CSEN 266 : Artificial Intelligence
HW2 : Report

Name: Inderpreet Kaur

ID:07700012331

Table of Contents:

Problem 1: Breadth-First Search.....2
Problem 2: Depth-First Search.....3
Problem 3: Uniform Cost Search.....4
Problem 4: Iterative Deepening Search.....6
Problem 5: A*.....7
Conclusions/Summary:.....9
Summary:.....10
Learnings:.....10
Group Info.....11

Problem 1: Breadth-First Search

```
def breadthFirstSearch(problem: SearchProblem) -> List[Directions]:
    """Search the shallowest nodes in the search tree first."""
    "* YOUR CODE HERE *"
    state = problem.getStartState()
    queue = util.Queue()
    queue.push(state)
    backtracking_map = {state: []}
    visited = set([state])

    while queue:
        node = queue.pop()
        if problem.isGoalState(node):
            return backtracking_map[node]

        for i in problem.getSuccessors(node):
            if i[0] not in visited:
                queue.push(i[0])
                visited.add(i[0])
                backtracking_map[i[0]] = backtracking_map[node] + [i[1]]
    return []
```

BFS code explanation:

- The function performs Breadth-First Search (BFS), which explores the search tree level by level, expanding the shallowest nodes first.
- It starts by getting the start state using problem.getStartState().
- A queue (FIFO) is created to store the states that need to be explored next.
- The start state is pushed into the queue, and its path is initialized as an empty list in the backtracking map.
- A visited set is created to keep track of states that have already been explored and prevent revisiting them.
- The algorithm enters a loop that continues until the queue is empty:
- Removes (pops) the first state from the queue.
- Check if the current state is the goal state. If yes, returns the list of actions (the path) stored in the backtracking map.
- Otherwise, retrieves all successors (neighboring states) of the current node.
- For each unvisited successor:
- Pushes it into the queue so it can be explored later.
- Marks it as visited.

- Records its path in the backtracking map by adding the new action to the parent's path.
- If the queue becomes empty and no goal is found, the function returns an empty list, meaning there is no valid path to the goal.

Frontier: Uses a queue (first-in, first-out). This means it explores the maze level by level.
Graph Search: Keeps a visited set to make sure each position is checked only once.
Successors: Finds all next possible moves using getSuccessors(). If a move hasn't been visited, it is added to the queue.
Goal Test: Each time a node is taken out of the queue, the code checks if it's the goal.
Solution Path: The path to each node is stored. When the goal is found, the path to it is returned.

Problem 2: Depth-First Search

```
def depthFirstSearch(problem: SearchProblem) -> List[Directions]:
    state = problem.getStartState()
    stack = util.Stack()
    stack.push(state)
    backtracking_map = {state: []}
    visited = set()

    while not stack.isEmpty():
        node = stack.pop()
        if problem.isGoalState(node):
            return backtracking_map[node]

        if node not in visited:
            visited.add(node)
            for i in problem.getSuccessors(node):
                if i[0] not in visited:
                    stack.push(i[0])
                    backtracking_map[i[0]] = backtracking_map[node] +
[i[1]]
    return []
```

Explanation of DFS code:

- Explores nodes as deep as possible before backtracking.

- Uses a stack (LIFO) to manage nodes for depth-first traversal.
- Starts from the initial state and stores paths using a backtracking map.
- Maintains a visited set to prevent revisiting the same states.
- Checks for the goal state after popping each node from the stack.
- For each unvisited successor, pushes it onto the stack and updates its path.
- Returns the action sequence once the goal is found, or an empty list if no path exists.
- The code was developed by adding the backtracking map, visited set, and early goal check, and simplifying the structure for clarity and correctness.

Frontier: Uses a stack (last-in, first-out), so it explores deep paths first.
Graph Search: Uses a visited set to avoid going back to already explored points.
Successors: Uses getSuccessors() to find next moves and pushes unvisited ones into the stack.
Goal Test: Checks if the node popped from the stack is the goal.
Solution Path: Stores the path taken to reach each node and returns it once the goal is found.

Problem 3: Uniform Cost Search

```
def uniformCostSearch(problem: SearchProblem) -> List[Directions]:
    """Search the node of least total cost first."""
    "* YOUR CODE HERE *"
    initial_state = problem.getStartState()
    priority_queue = util.PriorityQueue()
    # Push initial state into Priority Queue with priority 0
    priority_queue.push((initial_state, [], 0), 0)
    visited = {}

    while not priority_queue.isEmpty():
        curr_state, path, cost = priority_queue.pop()

        if problem.isGoalState(curr_state):
            return path

        # If the state is already visited with a lower cost before, skip
        if curr_state in visited and visited[curr_state] <= cost:
            continue
        visited[curr_state] = cost

        for next, action, step_cost in problem.getSuccessors(curr_state):
            new_cost = cost + step_cost
```

```
            if next not in visited or new_cost < visited[next]:
                new_path = path + [action]
                priority_queue.push((next, new_path, new_cost), new_cost)

    return []
```

Explanation of UCS code:

- The function performs Uniform Cost Search (UCS), which always expands the node that has the lowest total path cost so far.
  The algorithm begins by getting the initial state using problem.getStartState().
- A priority queue is used as the frontier, where each node's priority represents the total cost of reaching that node from the start.
- The start state is added to the priority queue with a priority of 0, since the cost to reach the start is zero.
  A visited dictionary keeps track of every explored state and the lowest cost found to reach it. This prevents revisiting a state through a more expensive path.

The algorithm runs while the priority queue is not empty:

- Removes (pops) the node with the lowest total cost from the priority queue.
  If this node is the goal, the algorithm immediately returns the path (sequence of actions) that leads to it.
- If the node has already been visited with a lower or equal cost, it is skipped to avoid redundant work.
- Otherwise, the current node and its total cost are stored in the visited dictionary.

For each successor (next state, action, and step cost):

- Calculates the new total cost as current cost + step cost.
  Creates a new path by adding the new action to the current path.
- Pushes the successor into the priority queue with the new total cost as its priority.
  If the queue becomes empty without finding the goal, the function returns an empty list, meaning no path exists.

Frontier: Uses a priority queue where each node is stored with its total path cost (g-value) as priority. The node with the lowest cost is expanded first.
Graph Search: Keeps a visited dictionary that records the lowest cost found for each explored state. If a state is reached again with a higher cost, it's skipped to avoid redundant work.
Successors: Generated using getSuccessors() which gives the next possible states, the action, and the step cost. Each successor is added to the priority queue with its new total cost (current cost + step cost) as the priority.

Goal Test: After popping a node from the queue, UCS checks isGoalState() to see if that node is the goal. If it is, the search stops immediately.
Solution Path: Each node carries the path of actions taken to reach it. When the goal is found, UCS returns this list of actions as the final solution.
G-values: The g-value (path cost) is updated as:new_cost = current_cost + step_cost
This makes sure nodes are expanded in increasing order of their total cost from the start.

Problem 4: Iterative Deepening Search

```
def IDSHelper(problem, maxDepth):
    state = problem.getStartState()
    stack = util.Stack()
    stack.push((state, 0))
    backtracking_map = {state: []}
    visited = set()

    depth = 0
    while not stack.isEmpty() and depth <= maxDepth:
        loc, depth = stack.pop()
        if problem.isGoalState(loc):
            return backtracking_map[loc]

        if loc not in visited and depth < maxDepth:
            visited.add(loc)
            for i in problem.getSuccessors(loc):
                if i[0] not in visited:
                    stack.push((i[0], depth + 1))
                    backtracking_map[i[0]] = backtracking_map[loc] +
[i[1]]

    return None

def iterativeDeepeningSearch(problem):
    """
    Perform DFS with increasingly larger depth. Begin with a depth of 1
and increment depth by 1 at every step.
    """
```

```
    "* YOUR CODE HERE *"
    depth = 1
    pathIDS = None
    while not pathIDS:
        pathIDS = IDSHelper(problem, depth)
        depth += 1
    return pathIDS
```

Explanation of IDS code:

- Iterative Deepening Search (IDS) repeatedly performs Depth-Limited Search (DFS with a cutoff depth), increasing the depth limit by 1 in each iteration until the goal is found.
- The IDSHelper() function performs a single depth-limited DFS up to a given maxDepth.
- A stack (LIFO) is used to manage nodes, ensuring depth-first traversal.
- The backtracking_map keeps track of the actions taken to reach each state so the final path can be reconstructed.
- A visited set prevents revisiting nodes within the same depth level.
- Each element in the stack stores a tuple (state, depth) so that the search can stop expanding nodes once the depth limit is reached.
- If the goal state is found within the current depth, the corresponding path from the backtracking_map is returned.
- If not found, the function returns None, signaling the need to increase the depth and search again.
- The iterativeDeepeningSearch() function starts with depth = 1 and calls IDSHelper() repeatedly, increasing the depth limit by 1 each time.
- The loop continues until a valid path is returned, ensuring completeness like BFS while maintaining DFS's low memory usage.
- This approach guarantees that the shallowest goal node is found first, combining the advantages of both DFS and BFS.

Frontier: Uses a stack like DFS but adds a depth limit that increases each round.
Graph Search: Runs DFS again and again, each time going one level deeper until it finds the goal.
Successors: Uses getSuccessors() to find next moves and pushes them onto the stack if within the depth limit.
Goal Test: Checks each popped node to see if it's the goal.
Solution Path: Keeps track of moves for each node and returns the path once the goal is reached.

Problem 5: A*

```
def aStarSearch(problem, heuristic=nullHeuristic):
    """
    A* Search: Expands nodes with the lowest total estimated cost f = g +
h.
    Returns a list of actions to reach the goal from the start state.
    """
    from util import PriorityQueue

    start_state = problem.getStartState()
    pq = PriorityQueue()
    pq.push((start_state, [], 0), heuristic(start_state, problem))

    #tracks the minimum known cost to reach each state
    visited_cost = {start_state: 0}

    while not pq.isEmpty():
        current_state, actions, current_cost = pq.pop()

        #skip if we already found a cheaper path to this state
        if current_cost > visited_cost.get(current_state, float("inf")):
            continue

        if problem.isGoalState(current_state):
            return actions

        #explore neighbors
        for next_state, direction, step_cost in
problem.getSuccessors(current_state):
            new_cost = current_cost + step_cost

            #only consider this path if it’s cheaper than any previously
found
            if new_cost < visited_cost.get(next_state, float("inf")):
                visited_cost[next_state] = new_cost
                total_estimated_cost = new_cost + heuristic(next_state,
problem)
                pq.push((next_state, actions + [direction], new_cost),
total_estimated_cost)
```

```
    return []
```

Explanation of A* code:

- A* Search expands nodes based on the lowest total estimated cost, computed as f = g + h, where
- g = actual cost from the start state to the current state,
- h = heuristic estimate from the current state to the goal.
- The priority queue stores tuples (state, path, cost) with priority equal to their total estimated cost f.
- The start state is obtained using problem.getStartState() and added to the queue with its heuristic value.
- A dictionary visited_cost tracks the minimum known cost (g) to reach each state to avoid revisiting more expensive paths.
- In each loop iteration, the algorithm pops the state with the lowest f value (most promising path) from the priority queue.
- If a cheaper path to the same state has already been found, the current path is skipped.
- If the current state satisfies the goal test, the corresponding sequence of actions (path) is returned.
- For each successor of the current node:
- Computes new_cost = current_cost + step_cost to update g.
- Calculates the total estimated cost f = g + h.
- Pushes the successor into the priority queue with f as the priority if this new path is cheaper.
- The loop continues until the goal is reached or the queue becomes empty (no solution).
- This approach ensures the shortest-cost path is found efficiently by combining actual and heuristic costs, balancing optimality and speed.

Frontier: Uses a priority queue that sorts nodes based on f = g + h,
where g = actual cost from start, and h = estimated distance to goal.
Graph Search: Stores the lowest known cost (g) for each node and updates it if a better path is found.
Successors: Uses getSuccessors() to get next moves and adds them to the queue using the combined cost (f).
Goal Test: When a node is taken from the queue, it checks if it is the goal.
Solution Path: Keeps a list of actions for each node and returns the full path once the goal is found.
g (Actual Cost): The value of g shows the total cost from the start state to the current node.
It is calculated by adding the step cost of each move to the current total cost. For example, if each move costs 1 and Pacman has taken 4 steps, then g = 4.
In the code, this is done using: new_cost = current_cost + step_cost
h (Heuristic Cost): The value of h is an estimated cost from the current node to the goal.

It does not use the real path but gives an idea of how close the node is to the goal.
f (Total Estimated Cost): The value of f is the sum of g and h.
It represents the total estimated cost of the path through the current node. f = g + h
In the code: total_estimated_cost = new_cost + heuristic(next_state, problem)

Conclusions/Summary:

The table shows comparison of Search Algorithm Performance Across Different Maze Sizes where:
M1: Number of nodes expanded (how many states the algorithm explored).
M2: Total path cost (the number of steps or cumulative cost in the final path).

Algorithm	tinyMaze (M1 / M2)	smallMaze (M1 / M2)	mediumMaze (M1 / M2)	bigMaze (M1 / M2)	Observations
BFS	15 / 8	92 / 19	269 / 68	620 / 210	Explores nodes level by level; guarantees the shortest path in terms of steps. Memory use grows with maze size, but it's complete and optimal.
DFS	15 / 10	59 / 49	146 / 130	390 / 210	Goes deep first; uses less memory but often explores long or dead-end paths. Not optimal for shortest path problems.
UCS	15 / 8	92 / 19	269 / 68	620 / 210	Expands nodes based on path cost. Guarantees the least-cost path. Same result as BFS in this grid since all step costs are equal.
IDS	64 / 8	1426 / 29	8580 / 70	60211 / 210	Performs DFS repeatedly with increasing depth. Finds the shortest path eventually but expands many nodes, making it slow on large mazes.
A*	14 / 8	53 / 19	221 / 68	549 / 210	Combines UCS's path-cost logic with heuristic guidance. Finds the optimal path with the fewest nodes expanded - most efficient overall.

Summary:

- BFS and UCS produced the same optimal paths in these mazes since each move cost is uniform.
- DFS was the fastest in smaller layouts but inefficient in large mazes due to unnecessary deep exploration.
- IDS achieved completeness like BFS but required significantly more expansions, proving less practical for large problems.
- A* outperformed all others - it reached the same optimal path cost as UCS but with the fewest nodes expanded, showing the power of heuristic-based search.
- Overall, A* provides the best balance between optimality, efficiency, and scalability, making it the preferred algorithm for real-world pathfinding problems.

Learnings:

From this assignment, I learned how different search algorithms work and how they perform in different situations. BFS finds the shortest path but uses more memory because it explores many nodes at the same level. DFS goes deep quickly and uses less memory, but it doesn't always find the best or shortest path. UCS looks for the lowest-cost path and always gives the best result, but it can take longer since it checks many nodes. IDS combines DFS and BFS ideas it uses little memory but repeats searches many times, so it's slower. A* search turned out to be the smartest. It uses both path cost and a heuristic to reach the goal faster while still finding the best path.

Overall, this experiment helped me understand the trade-offs between speed, memory, and accuracy. I learned that choosing the right search algorithm depends on the problem and that using a good heuristic (like in A*) can make searches much faster and more efficient.

Group Info

Group members: Smruthi Danda, Inderpreet Kaur, and Diola Dsouza

How did you divide the work?: We divided the work equally among all members. Initially, we spent time together understanding the Pacman environment, how search algorithms are implemented, and how the testing framework works.

Who did what?

- DFS & IDS: Implemented by Smruthi.
- BFS & UCS: Implemented by Diola.
- A* Search: Implemented by Inder.

What tools did you use?

We used Python 3, VS Code, and the Berkeley Pacman AI framework provided for the assignment. For testing and visualization, we used the Pacman simulator and autograder.py to validate each algorithm across different maze layouts.